INTELLIGENT FILE SYSTEM THREAT DETECTION AND EARLY WARNING USING BEHAVIOUR ANALYSIS AND AI-ASSISTED RISK ASSESSMENT

A PROJECT REPORT

Submitted by

NITHEESH K K 2116231901035

ARAVIND S 2116231901006

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**INTELLIGENT FILE SYSTEM THREAT DETECTION AND EARLY WARNING USING BEHAVIOUR ANALYSIS AND AI-ASSISTED RISK ASSESSMENT**" is the bonafide work of **"ARAVIND S (2116231901006), NITHEESH K K (2116231901035)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

SIGNATURE

Mrs. Seetha Lakshmi T,

SUPERVISOR

ASSISTANT PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 6.10.26

Internal Examiner

External Examiner

ABSTRACT

The increasing dependence on digital systems has resulted in a growing need for effective endpoint security mechanisms capable of identifying suspicious activities at an early stage. Traditional security solutions primarily rely on signature-based detection, which may have limitations when dealing with new, modified, or previously unknown threats. The proposed project, Intelligent File System Threat Detection and Early Warning Using Behaviour Analysis and AI-Assisted Risk Assessment, addresses this challenge through continuous monitoring and analysis of file-system behaviour.

The proposed system uses Java NIO WatchService to monitor selected directories and capture file-system activities such as file creation, modification, deletion, and renaming. The collected events are analysed using predefined behavioural indicators, including rapid file modifications, bulk deletions, mass file renaming, suspicious file extensions, and unusually high file activity. These indicators are evaluated using a weighted risk assessment mechanism to calculate an overall risk score and classify the detected activity into Low, Medium, or High risk levels.

When suspicious behaviour exceeds predefined thresholds, the system generates an early warning to notify the user. An AI-assisted threat assessment module further processes the structured security information to generate human-readable explanations of the detected behaviour and provide recommended response actions. The AI component is primarily intended to improve the interpretability of security alerts rather than replace the core behavioural detection mechanism.

The proposed system provides a lightweight and modular approach to endpoint threat monitoring by combining real-time file-system observation, behavioural analysis, risk assessment, early warning, and AI-assisted security explanation. The system can help users identify potentially harmful file-system behaviour at an early stage and provides a foundation for future integration of machine-learning-based anomaly detection, process-level monitoring, and automated security response mechanisms.

Keywords: File-System Security, Behaviour Analysis, Threat Detection, Early Warning, Risk Assessment, Endpoint Security, Java NIO WatchService, Artificial Intelligence, Cybersecurity.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head,** Department of Computer Science and Engineering (Cyber Security) and our project guide **Ms. T. Seethalakshmi., Assistant Professor,** for his encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in the successful completion of this project, and for their encouragement and support.

NITHEESH K K 2116231901035

ARAVIND S 2116231901006

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	vi
	LIST OF FIGURES	vii
1	**INTRODUCTION**	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVES	4
	1.4 SCOPE	5
	1.5 SECURITY REQUIREMENTS	6
	1.6 SDG ALIGNMENT	8
2	**LITERATURE REVIEW**	9
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	9
	2.2 RESEARCH GAPS	12
	2.3 PROPOSED SYSTEM	13
3	**SYSTEM DESIGN**	15
	3.1 SYSTEM ARCHITECTURE	15
	3.2 MODULE DESIGN	16
	3.3 DATA FLOW / UML DIAGRAMS	18
	3.4 THREAT MODEL AND ATTACK SURFACE	22
	3.5 SECURITY CONTROLS	23
4	**SYSTEM DESCRIPTION**	24

4.1 DATA COLLECTION 24
4.2 DATA PREPROCESSING 24
5 CONCLUSION 31
REFERENCES 33
APPENDIX 34

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SECURITY CONSIDERATIONS AND DESIGN RESPONSE	4
1.2	SDG ALIGNMENT	5
2.1	COMPARISON OF DETECTION APPROACHES	8
3.1	INPUTS AND OUTPUTS OF THE SYSTEM LAYERS	12
3.2	MODULE RESPONSIBILITIES	14
3.3	THREATS AND DETECTION RESPONSE	20
3.4	HARDWARE REQUIREMENTS	21
3.5	SOFTWARE REQUIREMENTS	21
4.1	MAPPING OF WATCHSERVICE NOTIFICATIONS TO EVENT TYPES	23
4.2	BEHAVIOURAL INDICATORS AND SUGGESTED DEFAULT THRESHOLDS	25
4.3	RISK LEVEL MAPPING	26
4.4	ILLUSTRATIVE CALCULATION OF THE RISK SCORE	27
4.5	TEST SCENARIOS AND EXPECTED BEHAVIOUR	29
4.6	PLANNED TEST CASES	30
A.1	MAJOR TECHNOLOGIES	35

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	LAYERED SYSTEM ARCHITECTURE	12
3.2	OVERALL DATA FLOW	13
3.3	LEVEL-0 DATA FLOW DIAGRAM	15
3.4	USE CASE DIAGRAM	16
3.5	ACTIVITY DIAGRAM OF THE MONITORING LOOP	17
3.6	SEQUENCE DIAGRAM	18
3.7	CLASS DIAGRAM	19
3.8	DASHBOARD LAYOUT	22
4.1	ILLUSTRATIVE RISK SCORE CONTRIBUTIONS	27
A.1	PROJECT DATA FLOW	36

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The increasing dependence on digital systems has resulted in a significant amount of sensitive information being stored and processed on endpoint devices. Files are continuously created, modified, deleted, and renamed during normal computer usage. However, abnormal patterns in these file-system operations can also indicate potentially harmful activities.

Traditional antivirus solutions primarily depend on signature-based detection mechanisms to identify known malicious software. Although signature-based approaches are effective against previously identified threats, they may have limitations when dealing with new, modified, or previously unknown threats. Behaviour-based security mechanisms provide an alternative approach by analysing the actions performed within a system rather than relying only on known signatures.

The proposed project focuses on monitoring file-system activity and identifying suspicious behavioural patterns. The system uses Java NIO WatchService to continuously monitor selected directories and capture file creation, modification, deletion, and renaming events.

The collected events are analysed using predefined behavioural indicators such as rapid file modifications, bulk deletions, mass renaming, suspicious file extensions, and unusually high file activity. A risk assessment mechanism assigns a risk score and classifies the activity into different severity levels. An AI-assisted module is then used to provide human-readable explanations and recommended actions.

The primary purpose of the system is to provide an early warning mechanism that can help users identify potentially harmful file-system behaviour before extensive damage occurs.

1.2 PROBLEM STATEMENT

Modern endpoint systems continuously perform a large number of file operations. Among these operations, certain behavioural patterns may indicate suspicious or potentially harmful activity. Traditional signature-based security mechanisms may not always identify new or modified threats that do not match previously known signatures.

Furthermore, technical security alerts may not clearly explain the reason for an alert or the possible implications of the detected activity. Users may therefore have difficulty

understanding whether an observed file-system event represents normal activity or a potential security threat.

The proposed system addresses these challenges by continuously monitoring file-system activity, analysing behavioural patterns, calculating risk levels, and providing early warnings accompanied by understandable explanations.

1.3 OBJECTIVES

The major objectives of the proposed system are:

1. To continuously monitor selected directories using Java NIO WatchService.
2. To detect file creation, modification, deletion, and renaming events.
3. To analyse file-system activity using predefined behavioural rules.
4. To identify suspicious patterns such as rapid modifications, bulk deletions, mass renaming, and unusually high file activity.
5. To identify files with predefined suspicious extensions as one of the behavioural indicators.
6. To calculate a risk score based on detected behavioural indicators.
7. To classify detected activity into Low, Medium, and High risk levels.
8. To generate early warning alerts when suspicious activity exceeds configured thresholds.
9. To provide AI-assisted explanations of detected security events.
10. To provide recommended response actions to help users understand and respond to potential threats.
11. To maintain logs of observed file-system activities and generated alerts.
12. To provide a user-friendly interface for monitoring system activity and security warnings.

1.4 SCOPE

The scope of the proposed system is focused on behavioural monitoring of file-system activity on an endpoint device.

The system monitors selected directories and captures relevant file-system operations including:

- File creation

- File modification
- File deletion
- File renaming

The system analyses these events within defined time windows to identify suspicious behavioural patterns. The system does not depend exclusively on known malware signatures. Instead, it evaluates observable file-system behaviour.

The system can identify patterns such as:

- Rapid modification of multiple files
- Bulk deletion of files
- Mass file renaming
- Suspicious file extensions
- Unusually high file activity

The system generates risk scores and early warnings based on predefined rules. An AI-assisted component provides explanations and recommended actions.

The project is intended as a security monitoring and early warning system and does not attempt to replace commercial antivirus or endpoint detection and response solutions.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

The proposed system should satisfy the following security considerations:

- **Real-Time Monitoring:** The system should continuously monitor the selected directory and capture relevant file-system events.
- **Event Integrity:** Captured events should contain important information such as event type, affected file, directory, and timestamp.
- **Secure Logging:** Security events and generated alerts should be recorded for later analysis.
- **Controlled Monitoring:** The system should monitor only directories selected by the user.
- **Risk Classification:** Detected behavioural indicators should be processed consistently using predefined risk rules.
- **Alert Reliability:** Warnings should be generated when suspicious activity exceeds the configured threshold.

- **AI Security:** The AI-assisted module should receive structured security information rather than unrestricted access to user files.
- **Privacy:** The system should avoid unnecessarily transmitting file contents or sensitive information to external AI services.
- **No Automatic Destructive Actions:** The initial system should focus on detection and warning rather than automatically deleting files or terminating processes.

Each of the above considerations is addressed by a specific design decision in the proposed system. Table 1.1 summarises how the security considerations are mapped to the corresponding design responses.

Table 1.1 Security Considerations and Design Response

Security Consideration	Design Response
Real-Time Monitoring	Java NIO WatchService registered on the selected directory delivers events as they occur.
Event Integrity	Every event is stored as a fixed record: type, file, directory, timestamp.
Secure Logging	Events and alerts are written to separate, timestamped log files.
Controlled Monitoring	Only the directory chosen by the user is registered for monitoring.
Risk Classification	A single weighted formula and fixed thresholds are applied to every time window.
Alert Reliability	Alerts are triggered by a configurable threshold on the risk score, not by single events.
AI Security	Only a structured summary (counts, indicators, file names) is passed to the AI module.
Privacy	File contents are never read or transmitted; the AI call can be disabled.
No Automatic Destructive Actions	The system only observes, warns and recommends; the user decides the response.

1.6 SDG ALIGNMENT

The proposed project supports the broader objective of creating safer and more reliable digital infrastructure.

Table 1.2 SDG Alignment

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 – Industry, Innovation and Infrastructure	Reliable and secure digital infrastructure	The system applies software and cybersecurity technologies to improve the security and reliability of digital infrastructure.
2	SDG 16 – Peace, Justice and Strong Institutions	Safer digital environments and protection of information systems	Improved cybersecurity mechanisms contribute to safer digital environments and protection of information systems.

1.7 METHODOLOGY OVERVIEW

The work was carried out in stages, each corresponding to a module of the final system:

1. Study of file-system behaviour and of the limitations of signature-based detection.
2. Identification of the file-system events that can be observed through Java NIO WatchService.
3. Definition of behavioural indicators that separate bursts of suspicious activity from normal usage.
4. Design of a weighted risk-scoring scheme and of thresholds for Low, Medium and High risk.
5. Design of the AI-assisted explanation module that works only on structured, privacy-conscious input.
6. Design of the dashboard and logging components, followed by planning of a controlled test strategy.

1.8 ORGANIZATION OF THE REPORT

The remainder of this report is organised as follows. Chapter 2 reviews existing security approaches, identifies the research gaps and presents the proposed system. Chapter 3 describes the system design, including the architecture, modules, data flow, class and sequence design, interface design, threat model and security controls. Chapter 4 describes data collection, preprocessing, the behavioural indicators, the risk-scoring algorithm, the AI-assisted module, logging and the evaluation plan. Chapter 5 concludes the report with its limitations and future enhancements.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Traditional endpoint security solutions commonly rely on signature-based detection to identify known malicious software. In such systems, files or applications are compared against previously identified malware signatures or patterns.

Although this approach can effectively identify known threats, it can be less effective against new or modified threats for which signatures are not yet available.

Behaviour-based security approaches address this limitation by analysing activities performed within a system. Instead of focusing exclusively on the identity of a malicious program, behaviour-based approaches examine patterns such as unusual file modifications, deletion activity, process behaviour, or abnormal system operations.

File-system behaviour is particularly useful for identifying suspicious activity because many harmful activities result in large numbers of file operations within a short period.

Existing security systems may also generate technical alerts that require security expertise to interpret. Therefore, providing meaningful explanations alongside security alerts can improve the usability of security monitoring systems.

2.1.1 SIGNATURE-BASED DETECTION

In signature-based detection, a security product keeps a database of byte patterns, hashes or other fingerprints of known malicious files. A file that matches an entry is flagged. The method is fast, produces few false positives for known samples and is easy to understand. Its weakness is that the database must be updated continuously. A modified version of a known threat, a newly written threat, or a threat that changes its own code can pass unnoticed until a signature is created for it.

2.1.2 HEURISTIC AND BEHAVIOUR-BASED DETECTION

Heuristic and behaviour-based approaches do not ask whether a program is already known. They ask what the program does. Typical observed actions include creating or modifying a large number of files, deleting data, changing file extensions, contacting remote hosts or altering system settings. A rule or a score is applied to these actions and an alert is raised when the behaviour looks abnormal. This approach can detect unknown threats, but it needs carefully

chosen thresholds. If they are too low, normal activities such as software installation or a backup job produce false alarms; if they are too high, real threats may be missed.

2.1.3 FILE-SYSTEM ACTIVITY AS A SECURITY SIGNAL

File-system activity is attractive as a security signal because almost every harmful action that affects user data must eventually touch files. Programs that encrypt, corrupt, wipe or steal data create, modify, rename or delete files, and they tend to do so in bursts that differ clearly from interactive human use. A person editing a document changes one file at a time, whereas a harmful process may change hundreds of files within seconds.

Operating systems expose file-system notifications through native mechanisms, for example inotify on Linux, ReadDirectoryChangesW on Windows and FSEvents on macOS. Java hides these differences behind the java.nio.file.WatchService interface, which allows a single implementation to register a directory and receive entry-create, entry-modify and entry-delete notifications. This makes WatchService a convenient and portable basis for a lightweight monitoring tool.

2.1.4 TYPICAL BEHAVIOUR OF HARMFUL FILE ACTIVITY

Although individual threats differ, harmful file activity often shows one or more of the following characteristics, and these characteristics are the basis of the indicators used in this project:

- A large number of files are modified within a very short period.
- Original files are deleted after new versions are written.
- Many files are renamed in sequence, often with a new or unusual extension appended.
- New files with unusual extensions appear in the monitored directory.
- The overall rate of file events is far above the normal rate for the directory.

2.1.5 EXPLAINABILITY IN SECURITY ALERTS

A common complaint about security tools is that an alert states a technical fact without explaining why it matters. An alert such as “threshold exceeded” is of limited use to a person who is not a security specialist. Explainable alerts describe which behaviour triggered the warning, why that behaviour is considered risky and what the user can do next. Language models are well suited to turning structured technical data into such readable explanations, provided that they are used as an assistant to the detection logic and are not given control over the detection decision itself. The proposed system follows this principle.

2.1.6 COMPARISON OF APPROACHES

Table 2.1 compares the main approaches discussed above with the proposed system.

Table 2.1 Comparison of Detection Approaches

Approach	Basis of Detection	Strengths	Limitations
Signature-based	Known patterns and hashes	Fast; very accurate for known threats	Cannot detect new or modified threats until a signature exists
Rule-based behaviour analysis	Observed actions compared with fixed rules	Detects unknown threats; transparent and easy to tune	Needs well-chosen thresholds; may miss slow attacks
Machine-learning anomaly detection	Deviation from a learned normal profile	Can adapt to different environments	Needs training data; harder to explain; higher cost
Proposed system	File-system events, weighted risk score, AI-assisted explanation	Lightweight; early warning; human-readable explanation; controlled scope	Rule-based; limited to monitored directories; no process-level view

2.2 RESEARCH GAPS

The following gaps are considered in the proposed work:

1. Traditional signature-based detection may have limitations when dealing with previously unknown or modified threats.
2. Some approaches focus primarily on malware identification rather than continuous monitoring of file-system behaviour.
3. Large volumes of file-system events can make it difficult for users to distinguish normal activity from suspicious activity.
4. Security alerts may provide technical information without sufficiently explaining the reason for the alert.
5. A combination of behavioural detection, risk scoring, early warning, and AI-assisted explanation can improve the interpretability of security monitoring.
6. There is a need for a lightweight monitoring approach that can continuously observe selected directories without requiring unrestricted access to the complete system.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system introduces an intelligent file-system monitoring framework that combines real-time event monitoring, behavioural analysis, risk assessment, early warning, and AI-assisted explanation.

The system uses Java NIO WatchService to monitor selected directories. Whenever a relevant file-system event occurs, the monitoring component records the event and forwards it to the Behaviour Analysis Layer.

The Behaviour Analysis Layer evaluates the event sequence and identifies suspicious patterns. The detected indicators are then passed to the Risk Assessment Layer, which calculates an overall risk score.

Based on predefined thresholds, the system classifies the activity as Low, Medium, or High risk. When the activity exceeds the configured warning threshold, an early warning is generated.

The structured security information is subsequently provided to the AI-assisted module, which generates a human-readable explanation and recommended response actions.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The proposed system aims to:

- Monitor file-system activity in real time.
- Detect abnormal file-system behaviour.
- Reduce dependence on signature-based detection.
- Identify suspicious behavioural patterns.
- Calculate security risk scores.
- Generate early warnings.
- Improve the interpretability of security alerts.
- Maintain security event logs.
- Provide users with recommended actions.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed approach provides several advantages:

- **Behaviour-based detection:** Focuses on activity patterns rather than only known signatures.
- **Real-time monitoring:** File-system events are captured as they occur.
- **Early warning:** Suspicious behaviour can be identified before extensive damage occurs.
- **Risk prioritization:** Events can be classified according to severity.
- **Human-readable explanations:** AI assistance helps users understand security alerts.
- **Modular design:** Monitoring, analysis, risk assessment, and AI components can be developed independently.
- **Extensibility:** Future machine-learning and process-monitoring capabilities can be added.
- **Controlled monitoring:** Users can select the directories to be monitored.

2.4 SUMMARY OF THE LITERATURE REVIEW

The review shows that signature-based detection remains useful but cannot be the only line of defence, that behaviour-based detection is a practical complement, and that file-system activity is one of the most direct behavioural signals available on an endpoint. It also shows that a gap exists between detection and understanding: users receive alerts but often do not understand them. The proposed system is positioned to address both aspects by combining rule-based behavioural detection with AI-assisted explanation, while keeping the monitoring scope controlled and the implementation lightweight.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed system consists of multiple layers that work together to identify suspicious file-system behaviour.

- **Presentation Layer:** Provides the user interface for selecting directories, starting and stopping monitoring, viewing events, viewing risk levels, receiving warnings, and reviewing logs.
- **File-System Monitoring Layer:** Uses Java NIO WatchService to monitor selected directories and capture CREATE, MODIFY, DELETE, and RENAME-related activity.
- **Behaviour Analysis Layer:** Evaluates event sequences and identifies suspicious patterns including rapid modification, bulk deletion, mass renaming, suspicious extensions, and high-frequency activity.
- **Risk Assessment and Early Warning Layer:** Calculates a weighted risk score and maps it to Low, Medium, or High risk. An early warning is generated when the configured threshold is exceeded.
- **AI-Assisted Explanation and Logging Layer:** Generates human-readable explanations and recommended actions while maintaining event and alert logs.

The layered structure of the system is shown in Figure 3.1. Each layer communicates only with the layer directly above or below it, which keeps the components loosely coupled and allows each of them to be tested and replaced independently.

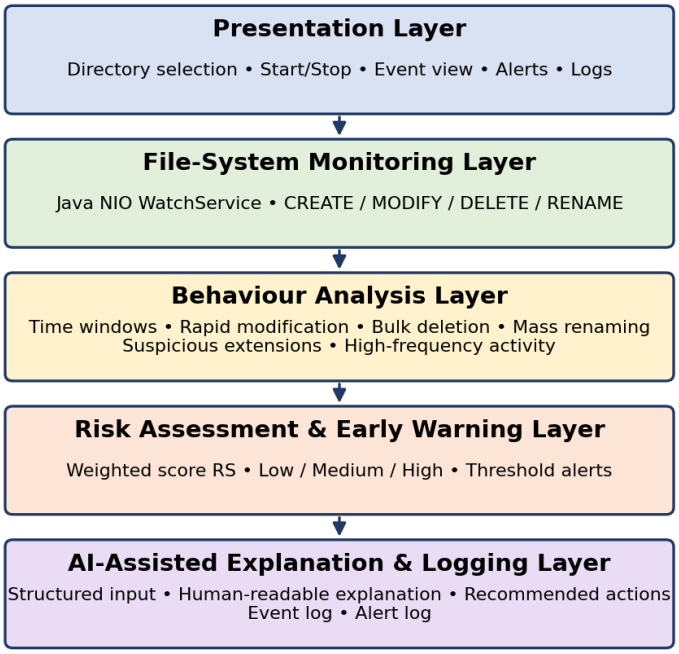


Figure 3.1 Layered System Architecture

Table 3.1 lists the input and output of each layer.

Table 3.1 Inputs and Outputs of the System Layers

Layer	Input	Output
Presentation	User actions; risk level, alerts and logs	Start/stop commands; displayed results
File-System Monitoring	Selected directory; WatchService notifications	Structured file events
Behaviour Analysis	Stream of file events	Behavioural indicators per time window
Risk Assessment and Early Warning	Behavioural indicators	Risk score, risk level, early warning
AI-Assisted Explanation and Logging	Structured security information	Explanation, recommended actions, log entries

3.1.1 DESIGN PRINCIPLES

- Separation of concerns: monitoring, analysis, scoring, explanation and display are separate components.
- Detection first, explanation second: the AI module never decides whether an alert is raised.

- Least privilege: only the directory chosen by the user is observed and file contents are never read.
- Transparency: every alert can be traced to indicators, counts and thresholds that are visible to the user.
- Configurability: windows, thresholds and weights are parameters, not fixed constants.

The overall data flow of the system is shown in Figure 3.2.

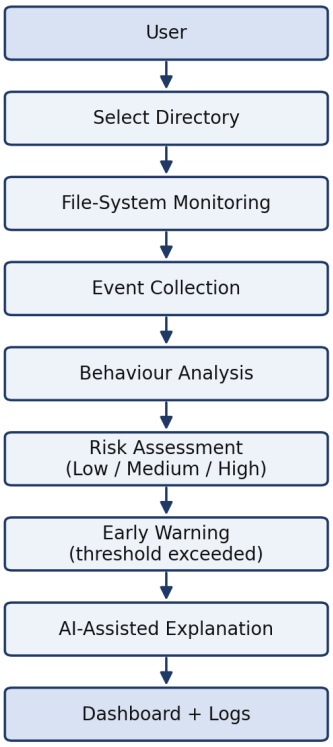


Figure 3.2 Overall Data Flow

3.2 MODULE DESIGN

- **Module 1: File-System Monitoring:** Uses Java NIO WatchService to observe selected directories and capture relevant file-system events.
- **Module 2: Event Logging:** Stores detected events along with event type, affected file, directory, timestamp, and risk status.
- **Module 3: Behaviour Analysis:** Analyses event sequences and identifies abnormal patterns rather than treating every individual file operation as a threat.
- **Module 4: Risk Assessment:** Calculates the risk score using predefined behavioural weights.
- **Module 5: Early Warning:** Generates an alert when the calculated risk reaches the configured threshold.

- **Module 6: AI-Assisted Threat Explanation:** Receives structured security information and generates understandable explanations and recommended actions.
- **Module 7: Dashboard and Logs:** Displays monitoring status, file events, risk scores, alerts, explanations, and historical logs.

Table 3.2 gives a compact view of the responsibilities of each module and the data that it exchanges with the others.

Table 3.2 Module Responsibilities

Module	Main Responsibility	Data Handled
File-System Monitoring	Register directory; receive notifications	Event kind, file name, path
Event Logging	Persist every observed event	Event record with timestamp and risk status
Behaviour Analysis	Maintain time window; count events; test indicators	Counts per event type; extension matches
Risk Assessment	Apply weights; map score to level	Indicator values; weights; thresholds
Early Warning	Raise alert when threshold is reached	Risk score; alert message
AI-Assisted Threat Explanation	Build input; obtain explanation and actions	Structured summary; explanation text
Dashboard and Logs	Display status, events, alerts and history	All of the above

3.3 DATA FLOW / UML DIAGRAMS

Level-0 Data Flow

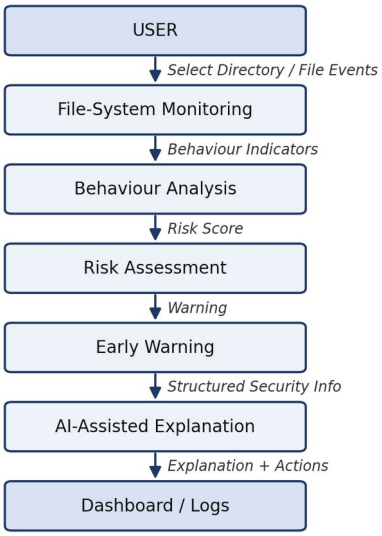


Figure 3.3 Level-0 Data Flow Diagram

Use Case Diagram Components

The primary actor is the User. Major use cases include:

- Select directory
- Start monitoring
- Stop monitoring
- View file-system events
- View risk score
- Receive alerts
- View AI explanation
- View logs

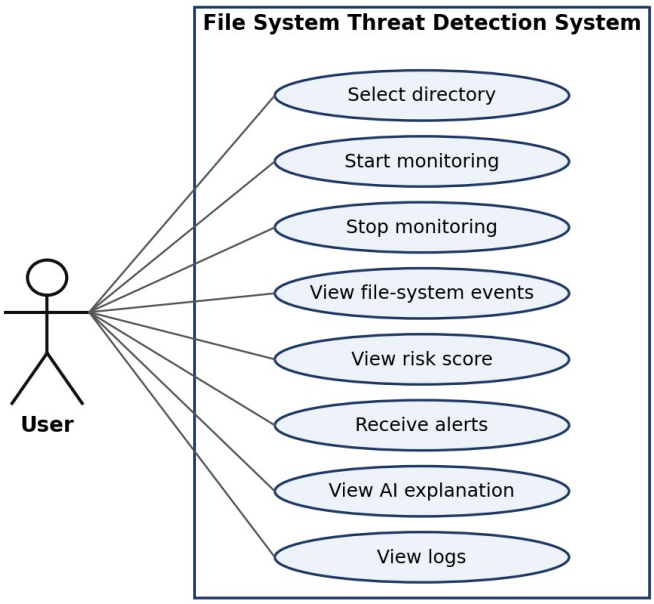


Figure 3.4 Use Case Diagram

Activity Diagram

The activity diagram in Figure 3.5 shows the continuous monitoring loop. After the directory is registered, the system waits for an event, records it, updates the current time window, evaluates the indicators and computes the risk score. Only when the score reaches the warning threshold does the system generate an early warning and request an AI-assisted explanation; otherwise the dashboard is simply updated and the loop continues.

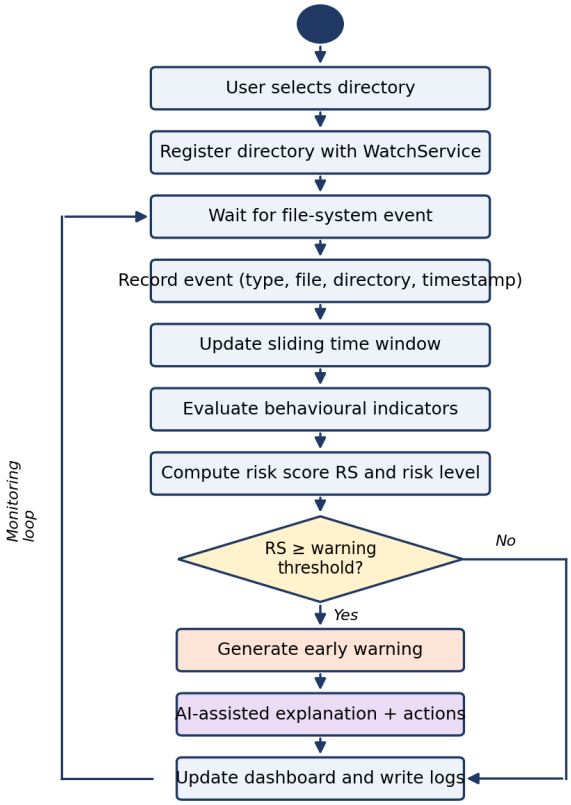


Figure 3.5 Activity Diagram of the Monitoring Loop

Sequence Diagram

Figure 3.6 shows the order of interactions between the main components for one pass through the monitoring loop, starting from the user's selection of a directory.

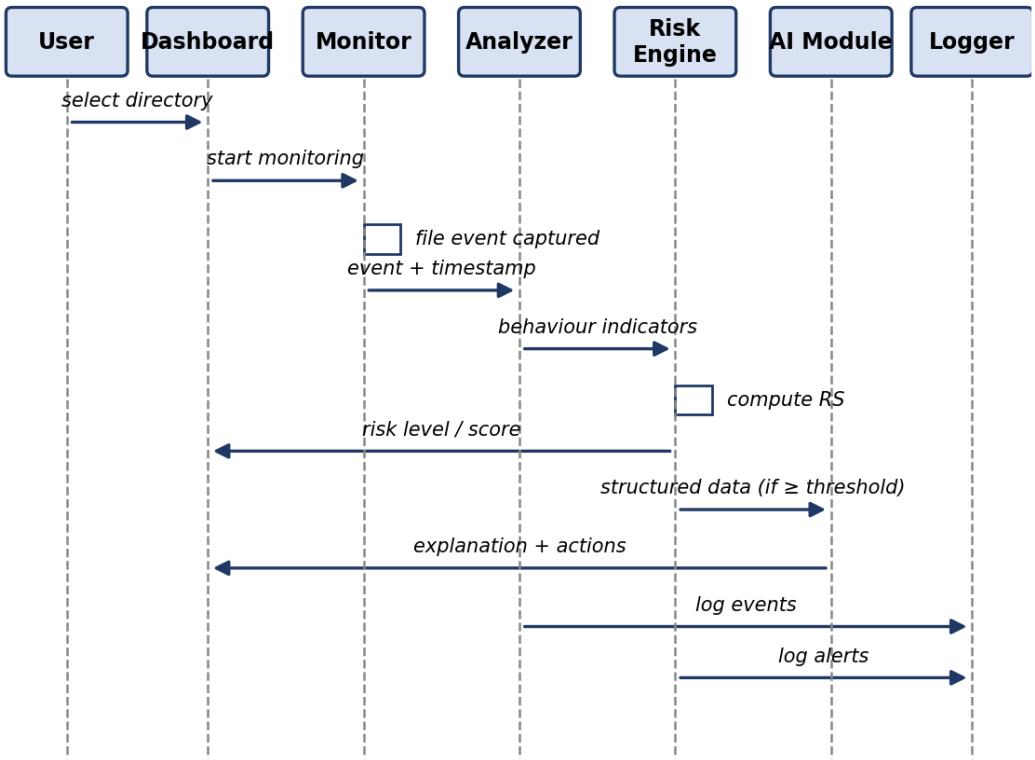


Figure 3.6 Sequence Diagram

Class Design

The system is implemented as a small set of cooperating classes, shown in Figure 3.7. FileEvent is a simple data holder. MonitorService reads notifications from WatchService and produces FileEvent objects. BehaviourAnalyzer keeps the sliding window and evaluates the indicators. RiskAssessor converts indicator values to a score and a level. AIExplainer prepares the structured input and obtains the explanation, EventLogger writes the logs and DashboardUI displays the results.

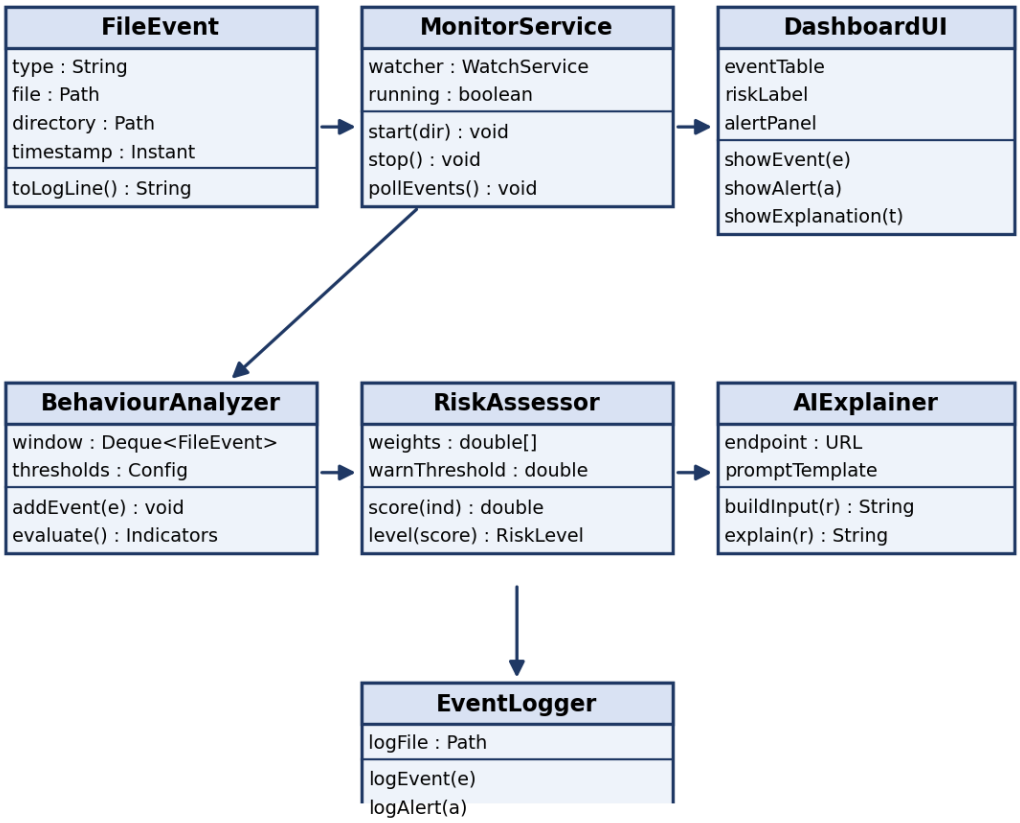


Figure 3.7 Class Diagram

3.4 THREAT MODEL AND ATTACK SURFACE

Potential threats considered by the system include:

- Rapid file modification
- Bulk file deletion
- Mass file renaming
- Suspicious file extensions
- High-frequency file activity

The primary attack surface includes monitored directories, file-system events, event-processing components, logging mechanisms, dashboard components, and the AI-assisted analysis interface.

Table 3.3 relates each threat to the behavioural indicator used to detect it and to the system response.

Table 3.3 Threats and Detection Response

Threat	Detecting Indicator	System Response
Rapid file modification	Number of MODIFY events in the time window	Adds to risk score; may trigger early warning
Bulk file deletion	Number of DELETE events in the time window	Adds to risk score; may trigger early warning
Mass file renaming	Number of paired delete/create events with new names	Adds to risk score; may trigger early warning
Suspicious file extensions	Files whose extension matches the configured list	Adds to risk score; file names passed to AI module
High-frequency file activity	Total events per time window	Adds to risk score; may trigger early warning

The attack surface also includes the system's own components. The logging files could be modified by a local attacker, the AI interface could receive over-long or manipulated input, and the dashboard could display misleading text. These risks are reduced by keeping the AI input structured and limited, by treating AI output only as advice, and by recommending that the log directory is placed outside the monitored directory.

3.5 SECURITY CONTROLS

The proposed system incorporates:

- Real-time monitoring
- Behavioural detection
- Risk scoring
- Early warning alerts
- Security event logging
- Controlled AI input
- Directory-level access control
- Privacy-conscious event processing

3.6 HARDWARE AND SOFTWARE REQUIREMENTS

The system is lightweight and does not require special hardware. Table 3.4 and Table 3.5 list the suggested minimum requirements for development and use.

Table 3.4 Hardware Requirements

Component	Suggested Requirement
Processor	Dual-core processor, 2 GHz or higher
Memory (RAM)	4 GB minimum; 8 GB recommended
Storage	500 MB free space for application and logs
Display	1366 × 768 resolution or higher
Network	Internet connection for the AI-assisted module (optional)

Table 3.5 Software Requirements

Component	Suggested Requirement
Operating system	Windows, Linux or macOS with a supported Java runtime
Programming language	Java (JDK 17 or later)
Monitoring API	Java NIO WatchService (java.nio.file)
User interface	Java GUI framework such as Swing or JavaFX
AI service	Language-model API accessed over HTTPS
Log storage	Plain-text or CSV log files
Development tools	Java IDE (for example IntelliJ IDEA, Eclipse or NetBeans)

3.7 USER INTERFACE DESIGN

The dashboard is designed so that the state of the system can be understood at a glance. A top bar lets the user choose the directory and start or stop monitoring. The left panel shows a live table of file events. The right panel shows the current risk level, using a colour for each level, and, when a warning is raised, the early warning message together with the AI-assisted explanation and recommended actions. A log view allows earlier events and alerts to be reviewed. A sketch of the main screen is given in Figure 3.8.

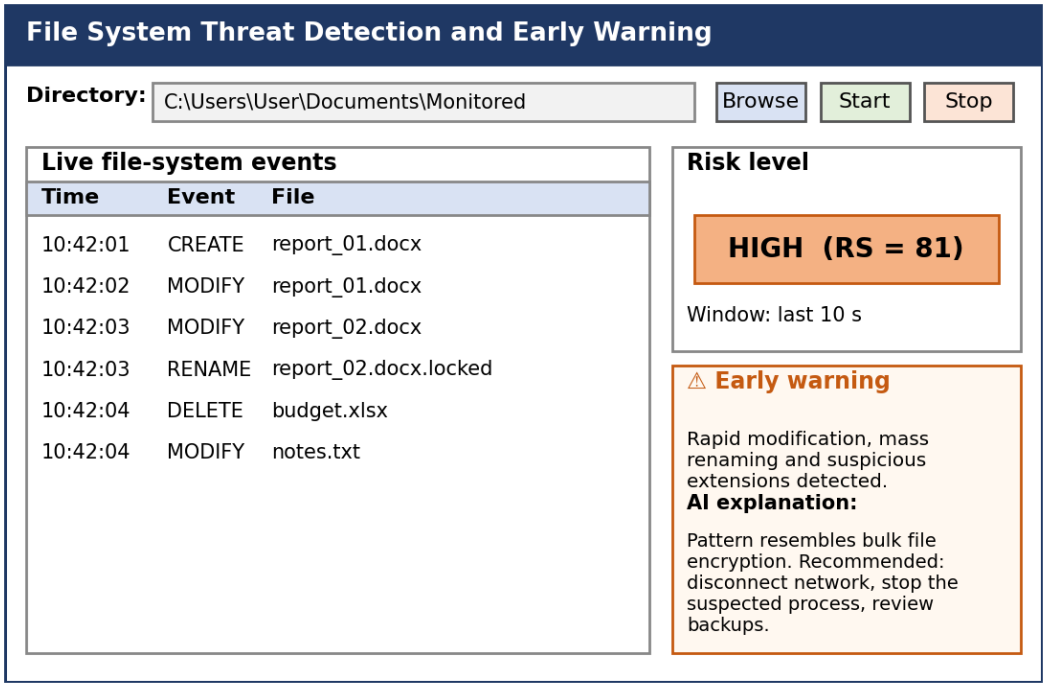


Figure 3.8 Dashboard Layout

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The system collects file-system events using Java NIO WatchService. The primary events collected are:

- File creation
- File modification
- File deletion
- File renaming

Each event can be represented as:

$$e_i = \{\text{type}, \text{file}, \text{directory}, \text{timestamp}\}$$

where each parameter represents the event type, affected file, monitored directory, and timestamp.

Java NIO WatchService reports three kinds of entry events, ENTRY_CREATE, ENTRY_MODIFY and ENTRY_DELETE, plus an OVERFLOW notice when events are lost because they arrive faster than they are read. A rename is not reported as a separate kind; it appears as an ENTRY_DELETE for the old name followed closely by an ENTRY_CREATE for the new name. The monitoring component therefore pairs such events that occur within a short interval and records them as a RENAME. Table 4.1 shows how WatchService notifications are mapped to the event types used in this report.

Table 4.1 Mapping of WatchService Notifications to Event Types

WatchService Notification	Recorded Event Type
ENTRY_CREATE	CREATE
ENTRY_MODIFY	MODIFY
ENTRY_DELETE	DELETE
ENTRY_DELETE followed closely by ENTRY_CREATE	RENAME
OVERFLOW	Noted in the log; the current window is marked as incomplete

WatchService registers a single directory and does not automatically include its subdirectories. To monitor a directory tree, the system walks the tree once and registers every subdirectory, and registers newly created subdirectories as they appear. A simplified outline of the monitoring loop is shown below.

```
WatchService watcher = FileSystems.getDefault().newWatchService();
Path dir = Paths.get(selectedDirectory);
dir.register(watcher,
    StandardWatchEventKinds.ENTRY_CREATE,
    StandardWatchEventKinds.ENTRY_MODIFY,
    StandardWatchEventKinds.ENTRY_DELETE);

while (running) {
    WatchKey key = watcher.take();
    for (WatchEvent<?> ev : key.pollEvents()) {
        if (ev.kind() == StandardWatchEventKinds.OVERFLOW) continue;
        Path file = dir.resolve((Path) ev.context());
        queue.add(new FileEvent(ev.kind().name(), file, dir, Instant.now()));
    }
    if (!key.reset()) break;
}
```

4.2 DATA PREPROCESSING

The preprocessing stage includes:

1. Identifying the event type.
2. Recording the affected file.
3. Recording the timestamp.
4. Grouping events within a defined time window.
5. Counting relevant event types.
6. Identifying suspicious extensions.
7. Detecting unusually high event frequency.

Grouping uses a sliding time window of configurable length (a default of 10 seconds is suggested). Events older than the window are removed, so the counts always describe recent behaviour. Normal activity, such as saving a document, produces very few events per window. A harmful burst produces many events of the same kind in a short time, which makes the window counts a useful input for the indicators.

4.3 ALGORITHMS / METHODOLOGY

A. File-System Event Collection

The system continuously monitors selected directories using Java NIO WatchService. The event sequence is represented as:

$$E = \{e_1, e_2, \ldots, e_n\}$$

B. Behavioural Analysis

The behavioural indicators are represented as:

$$B = \{b_1, b_2, \ldots, b_m\}$$

The system evaluates rapid modifications, bulk deletions, mass renaming, suspicious extensions, and high-frequency activity.

C. Risk Score Computation

The overall risk score is calculated as:

$$RS = \sum_{i=1 \text{ to } m} w_i b_i$$

The resulting score is mapped to Low, Medium, or High risk.

D. AI-Assisted Threat Assessment

The AI-assisted module receives structured information about the detected behaviour and generates a human-readable explanation and recommended response actions.

E. Behavioural Indicators and Thresholds

The system evaluates five behavioural indicators. Each indicator has a trigger threshold that is expressed as a count within one time window. The values in Table 4.2 are suggested defaults for a 10-second window; they are configurable and are intended to be calibrated during testing.

Table 4.2 Behavioural Indicators and Suggested Default Thresholds

Indicator	Definition	Weight (w)	Threshold (T)
Rapid file modification	Number of MODIFY events in the window	25	20 events
Bulk file deletion	Number of DELETE events in the window	25	15 events
Mass file renaming	Number of RENAME events in the window	20	15 events
Suspicious file extension	Number of files whose extension matches the configured list	15	3 files
High-frequency activity	Total number of events of all kinds in the window	15	50 events

The suspicious extension list is configurable. Typical entries include extensions that are commonly appended by file-encrypting programs, such as .locked, .encrypted, .crypt and .enc. A match is only one indicator and never causes an alert on its own unless it is accompanied by other activity that raises the score to the threshold.

F. Risk Score Computation in Detail

For each indicator, the observed count n in the current window is normalised against its threshold T to give a value b between 0 and 1:

$$b_i = \min(1, n_i / T_i)$$

The weights are chosen so that they add up to 100, which makes the risk score RS a value between 0 and 100. The score is then mapped to a risk level as shown in Table 4.3.

Table 4.3 Risk Level Mapping

Risk Score (RS)	Risk Level	System Behaviour
0 – 39	Low	Normal activity; dashboard shows green status
40 – 69	Medium	Elevated activity; dashboard highlights the indicators
70 – 100	High	Early warning generated; AI-assisted explanation requested

The warning threshold (default 70) is a configuration value, so that a more cautious user can choose to receive warnings at the Medium level as well.

A simplified implementation of the scoring step is shown below.

```
double score = 0;
for (Indicator ind : indicators) {
    double b = Math.min(1.0, ind.observed() / (double) ind.threshold());
    score += ind.weight() * b;
}
RiskLevel level = score >= 70 ? RiskLevel.HIGH
                : score >= 40 ? RiskLevel.MEDIUM : RiskLevel.LOW;
```

G. Worked Example

Three illustrative windows are used to show how the scheme behaves. They are constructed examples to explain the calculation and are not measured results. In the first window, a user saves a few documents; in the second, a backup tool touches many files; in the third, a burst of modifications, deletions, renames and suspicious extensions occurs. Table 4.4 gives the counts and Figure 4.1 shows the contribution of each indicator to the resulting score.

Table 4.4 Illustrative Calculation of the Risk Score

Indicator	Normal use	Moderate activity	Suspicious burst
Modifications (T = 20)	3 → 3.75	14 → 17.5	30 → 25.0
Deletions (T = 15)	0 → 0	9 → 15.0	6 → 10.0
Renames (T = 15)	0 → 0	6 → 8.0	12 → 16.0
Suspicious extensions (T = 3)	0 → 0	0 → 0	4 → 15.0
Total events (T = 50)	3 → 0.9	30 → 9.0	60 → 15.0
Risk score RS	4.7 (Low)	49.5 (Medium)	81.0 (High)

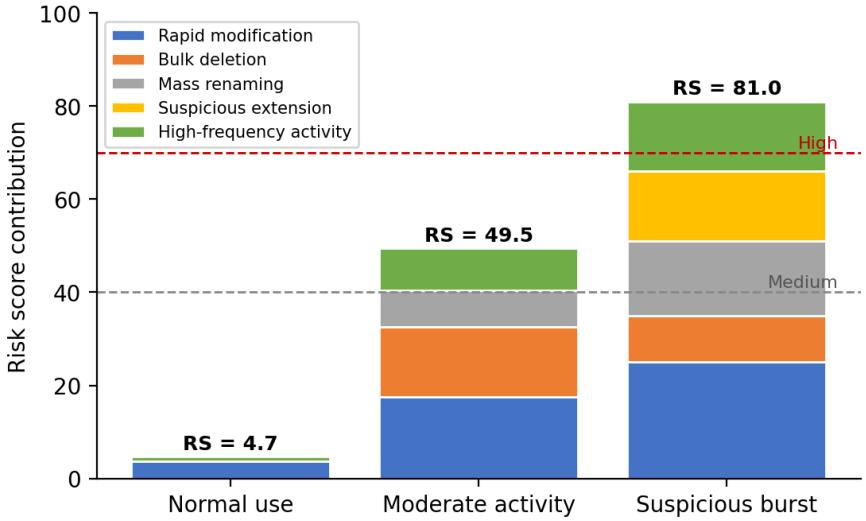


Figure 4.1 Illustrative Risk Score Contributions

In the third window every indicator contributes, and the total of 81 exceeds the warning threshold, so an early warning is generated. In the second window the score reaches the Medium level but stays below the warning threshold. This shows that the weighted combination reacts to the overall pattern and not to a single count.

H. Early Warning Generation

When the score reaches the warning threshold, the early warning component creates an alert that contains the time, the monitored directory, the risk score and level, the indicators that contributed most, and a short list of affected file names. To avoid a flood of identical alerts, a repeated alert for the same directory is suppressed for a configurable cool-down period unless the risk level increases.

I. AI-Assisted Module in Detail

The AI-assisted module receives a compact structured summary of the alert. File contents are never included. An example of the structured input is shown below.

```
{
  "riskLevel": "HIGH",
  "riskScore": 81,
  "windowSeconds": 10,
  "directory": "Monitored",
  "indicators": [
    {"name": "RapidModification", "observed": 30, "threshold": 20},
    {"name": "BulkDeletion", "observed": 6, "threshold": 15},
    {"name": "MassRenaming", "observed": 12, "threshold": 15},
    {"name": "SuspiciousExtension", "observed": 4, "threshold": 3}
  ],
  "sampleFiles": ["report_02.docx.locked", "budget.xlsx.locked"]
}
```

The module combines this summary with a fixed instruction asking the language model to explain, in simple language, why the behaviour is considered risky, how serious it is and what the user should do next. The answer is displayed in the dashboard together with the alert. If the AI service is not reachable or is disabled, the system displays a built-in template message for each indicator so that the early warning is never delayed or lost. This keeps the core detection independent of the AI component.

Recommended response actions are advisory. Examples include disconnecting the device from the network, identifying and closing the program that is generating the activity, checking recent backups, and avoiding the opening or saving of further files in the monitored directory until the cause is understood.

J. Logging

Two logs are maintained. The event log stores every captured event as one line containing the timestamp, event type, file, directory and current risk status. The alert log stores each early warning with its score, level, contributing indicators and the explanation shown to the user. Both logs are append-only text files that can be reviewed in the dashboard or opened in other tools. Because the logs are themselves files, it is recommended that they are stored outside the monitored directory so that they do not generate events in the monitored window.

4.4 RESULTS AND DISCUSSION

The system will be evaluated using controlled normal and suspicious file-system activity scenarios.

Table 4.5 Test Scenarios and Expected Behaviour

Test Scenario	Expected Behaviour
Normal file creation	Low risk
Normal file modification	Low risk
Normal file deletion	Low risk
Rapid file modification	Increased risk
Bulk deletion	Increased risk
Mass renaming	Increased risk
Suspicious extension	Risk indicator
High-frequency activity	Increased risk

The final report should include actual experimental values for detection rate, false-positive rate, detection time, alert-generation time, and risk-classification performance after implementation and testing.

Test Environment and Method

The evaluation is planned in a controlled environment in which a test directory is created on a single computer. A small script generates two kinds of activity. Normal activity imitates ordinary use: creating, editing and deleting a small number of files at human speed. Suspicious activity imitates the behaviours in Table 4.5: modifying many files in quick succession, deleting many files, renaming many files, creating files with suspicious extensions and producing a very high rate of events. The scripts do not contain any harmful code and operate only on dummy files in the test directory.

Evaluation Metrics

The following metrics will be used to evaluate the implemented system:

- **Detection rate:** The proportion of suspicious scenarios for which a High-risk alert is generated.
- **False-positive rate:** The proportion of normal scenarios for which a warning is generated.
- **Detection time:** The time between the start of the suspicious activity and the moment the risk score first reaches the warning threshold.
- **Alert-generation time:** The time between reaching the threshold and the display of the early warning.

- **Risk-classification performance:** The agreement between the risk level produced by the system and the level expected for each scenario.

These metrics are expressed in the usual way. If TP is the number of suspicious scenarios correctly flagged, FN the number missed, FP the number of normal scenarios wrongly flagged and TN the number correctly left unflagged, then the detection rate is TP / (TP + FN) and the false-positive rate is FP / (FP + TN).

Planned Test Cases

Table 4.6 Planned Test Cases

ID	Scenario	Activity Generated	Expected Result
T1	Normal editing	Create and save 3–5 files at human speed	Low risk; no warning
T2	Normal housekeeping	Delete 2–3 files	Low risk; no warning
T3	Rapid modification	Modify 30 files within 10 seconds	Rapid-modification indicator at full value; risk increased
T4	Bulk deletion	Delete 20 files within 10 seconds	Bulk-deletion indicator at full value; risk increased
T5	Mass renaming	Rename 20 files within 10 seconds	Mass-renaming indicator at full value; risk increased
T6	Suspicious extension	Create 5 files with entries from the extension list	Suspicious-extension indicator triggered
T7	High-frequency activity	Generate 80 mixed events within 10 seconds	High-frequency indicator at full value; risk increased
T8	Combined burst	Modify, rename and add suspicious extensions to 30 files	High risk; early warning and explanation shown
T9	AI unavailable	Repeat T8 with the AI service disabled	Early warning shown with built-in template explanation
T10	Backup-like activity	Copy and modify a large number of files slowly	Expected to stay at Low or Medium; used to study false positives

Discussion

The final report should include actual experimental values for detection rate, false-positive rate, detection time, alert-generation time, and risk-classification performance after implementation and testing. At this stage the discussion is therefore limited to the expected behaviour of the

design. Because the score depends on counts within a short window, slow and distributed activity is expected to produce lower scores than a rapid burst, and activities such as software installation or backups are the most likely sources of false positives. These cases will be examined during testing and the thresholds and weights will be adjusted accordingly.

4.5 LIMITATIONS AND CHALLENGES

- The system observes only the directories chosen by the user and cannot see activity elsewhere on the device.
- WatchService does not identify which process caused an event, so the source of suspicious activity must be found by the user.
- Renames are inferred from paired delete and create events and may occasionally be mis-paired when many events arrive together.
- If events are produced faster than they are read, the OVERFLOW notice indicates that some events were lost for that window.
- The detection depends on rule thresholds that must be calibrated; poor choices can lead to false positives or missed detections.
- Very slow activity that stays under the window thresholds may not raise the score.
- The AI-assisted explanation depends on an external service and is advisory only; its wording may vary between runs.

CHAPTER 5

CONCLUSION

This project presents an Intelligent File System Threat Detection and Early Warning system designed to identify suspicious file-system activities and provide timely security alerts. The proposed system integrates real-time file-system monitoring using Java NIO WatchService with behavioural analysis, risk assessment, and AI-assisted threat explanation.

Unlike traditional security mechanisms that primarily depend on known malware signatures, the proposed approach focuses on identifying abnormal patterns in file-system activity.

The system analyses file creation, modification, deletion, and renaming events to identify behavioural indicators such as rapid file modifications, bulk deletions, mass renaming, suspicious file extensions, and unusually high file activity.

The detected behavioural indicators are evaluated using a risk assessment mechanism to classify activities into different severity levels and generate early warnings when suspicious behaviour exceeds predefined thresholds. The AI-assisted module further improves the interpretability of security alerts by providing human-readable explanations and recommended response actions.

The modular architecture provides flexibility for future improvements and integration with more advanced security mechanisms. Future development can extend the system with machine-learning-based anomaly detection, process-level monitoring, automated response mechanisms, and broader platform support.

5.1 LIMITATIONS OF THE PRESENT WORK

The present work is limited to rule-based analysis of file-system events within user-selected directories. It does not identify the process responsible for an event, does not inspect file contents and does not take automatic action. The numerical performance of the system has to be established through the planned experiments before conclusions about accuracy can be drawn.

5.2 FUTURE ENHANCEMENTS

- Machine-learning-based anomaly detection that learns a normal activity profile for each monitored directory.
- Process-level monitoring to link suspicious file activity to the program that caused it.

- Additional indicators such as file-content entropy change and creation of ransom-note-like files.
- Optional automated response mechanisms, such as suspending a process or isolating the device, with user confirmation.
- Support for monitoring several directories and external drives at the same time.
- A local, offline language model option so that no information needs to leave the device.
- Packaged installers and broader platform testing.

REFERENCES

[1] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, "A survey on automated dynamic malware-analysis techniques and tools," *ACM Computing Surveys*, vol. 44, no. 2, Art. 6, 2012.

[2] A. Moser, C. Kruegel, and E. Kirda, "Limits of static analysis for malware detection," in *Proc. 23rd Annual Computer Security Applications Conf. (ACSAC)*, 2007, pp. 421-430.

[3] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. C. Lupu, "Automated dynamic analysis of ransomware: Benefits, limitations and use for detection," arXiv:1609.03020, 2016.

[4] A. Kharraz, W. Robertson, D. Balzarotti, L. Bilge, and E. Kirda, "Cutting the Gordian knot: A look under the hood of ransomware attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, LNCS 9148, Springer, 2015, pp. 3-24.

[5] A. Kharraz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "UNVEIL: A large-scale, automated approach to detecting ransomware," in *Proc. 25th USENIX Security Symp.*, 2016, pp. 757-772.

[6] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and drop it): Stopping ransomware attacks on user data," in *Proc. IEEE 36th Int. Conf. Distributed Computing Systems (ICDCS)*, 2016, pp. 303-312.

[7] A. Continella *et al.*, "ShieldFS: A self-healing, ransomware-aware filesystem," in *Proc. 32nd Annual Conf. Computer Security Applications (ACSAC)*, 2016, pp. 336-347.

[8] E. Kolodenker, W. Koch, G. Stringhini, and M. Egele, "PayBreak: Defense against cryptographic ransomware," in *Proc. ACM Asia Conf. Computer and Communications Security (ASIACCS)*, 2017, pp. 599-611.

[9] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135-1144.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765-4774.

[11] A. B. Arrieta *et al.*, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82-115, 2020.

[12] K. Kent and M. Souppaya, *Guide to Computer Security Log Management*, NIST Special Publication 800-92, 2006.

[13] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, 1948. (Use this if you apply entropy to detect encrypted or high-randomness files.)

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001; and F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011. (Use these only if your system includes an ML classifier.)

[15] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Rev. 2, 2012.

[16] M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. San Francisco, CA, USA: No Starch Press, 2012.

[17] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015. (This supports your SDG alignment section, probably SDG 9 and SDG 16.)

APPENDIX

Appendix A – Major Technologies

Table A.1 Major Technologies

Technology	Purpose
Java	Core implementation
Java NIO WatchService	Real-time file-system monitoring
Java GUI Framework	User interface/dashboard
Rule-Based Analysis	Behaviour detection
Risk Scoring	Threat prioritization
AI API	Human-readable threat explanation
File Logging	Security event storage

Appendix B – Major Modules

1. File-System Monitoring
2. Event Logging
3. Behaviour Analysis
4. Risk Assessment
5. Early Warning
6. AI-Assisted Threat Explanation
7. Dashboard
8. Security Logs

Appendix C – Project Data Flow

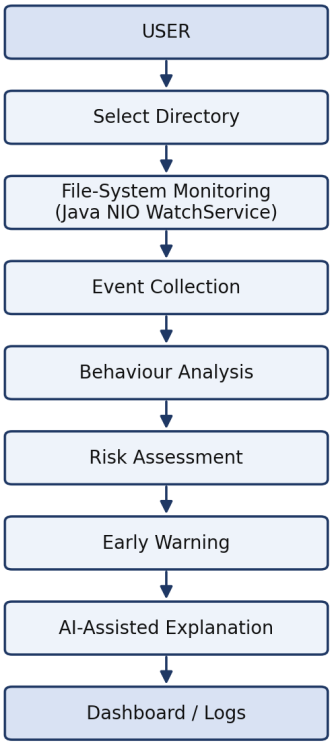


Figure A.1 Project Data Flow